

Algorithmen und Programmierung

Informatik · Klasse 7 · Lernkontrollen

Inhaltsverzeichnis

Leistungskontrolle 2 - Algorithmen und Scratch	2
Aufgabe 1 - Algorithmus beurteilen (6 Punkte)	2
Aufgabe 2 - Grundstrukturen (6 Punkte)	2
Aufgabe 3 - Scratch-Programm lesen (6 Punkte)	2
Aufgabe 4 - Schleifen (4 Punkte)	2
Aufgabe 5 - Debugging (4 Punkte)	2
Aufgabe 6 - Mini-Entwurf (4 Punkte)	3
Lösung - Leistungskontrolle 2 Algorithmen und Scratch	4
Aufgabe 1 - 6 Punkte	4
Aufgabe 2 - 6 Punkte	4
Aufgabe 3 - 6 Punkte	4
Aufgabe 4 - 4 Punkte	4
Aufgabe 5 - 4 Punkte	4
Aufgabe 6 - 4 Punkte	5

Leistungskontrolle 2 - Algorithmen und Scratch

Name: _____ Klasse: _____

Arbeitszeit: 45 Minuten

Gesamt: 30 Punkte

Aufgabe 1 - Algorithmus beurteilen (6 Punkte)

Ein Algorithmus lautet:

1. Starte.
2. Rechne etwas aus.
3. Zeige das Ergebnis.

Nenne drei Probleme dieser Beschreibung und verbessere den Algorithmus so, dass zwei eingegebene Zahlen addiert werden.

Aufgabe 2 - Grundstrukturen (6 Punkte)

Ordne jeweils **Sequenz**, **Entscheidung** oder **Wiederholung** zu und begründe kurz.

1. „Gehe fünfmal einen Schritt.“
2. „Wenn die Taste gedrückt ist, springe.“
3. „Frage nach dem Namen und gib anschließend eine Begrüßung aus.“

Aufgabe 3 - Scratch-Programm lesen (6 Punkte)

Ein Programm arbeitet sinngemäß so:

```
wenn grüne Flagge angeklickt
setze Punkte auf 0
frage „Wie viel ist 3 + 4?“ und warte
wenn <Antwort = 7> dann
    ändere Punkte um 1
    sage „richtig“
sonst
    sage „falsch“
```

1. Welche Eingabe erhält das Programm? (1 P)
2. Welche Bedingung wird geprüft? (1 P)
3. Was wird bei richtiger Antwort ausgegeben? (1 P)
4. Welchen Wert besitzt Punkte danach? (1 P)
5. Warum wird Punkte am Anfang auf 0 gesetzt? (2 P)

Aufgabe 4 - Schleifen (4 Punkte)

Eine Figur soll ein Quadrat zeichnen. Beschreibe einen möglichst kurzen Algorithmus mit einer Schleife.

Nenne einen Vorteil der Schleife gegenüber viermaligem Kopieren derselben Blöcke.

Aufgabe 5 - Debugging (4 Punkte)

Ein Punktespiel soll bei jedem Klick einen Punkt hinzufügen. Der Punktestand ändert sich nicht.

Nenne zwei mögliche Fehlerursachen und beschreibe jeweils einen passenden Test.

Aufgabe 6 - Mini-Entwurf (4 Punkte)

Plane ein Scratch-Programm, das nach einer Zahl fragt und ausgibt, ob die Zahl größer als 20 ist.
Notiere den Ablauf in verständlichen Schritten oder als einfaches Ablaufdiagramm.

Lösung - Leistungskontrolle 2 Algorithmen und Scratch

Aufgabe 1 - 6 Punkte

Mögliche Probleme:

- „Rechne etwas aus“ ist nicht eindeutig.
- Eingaben fehlen.
- Rechenoperation fehlt.
- Ausgabeschritt ist nicht konkret genug.

Verbesserung:

1. Starte das Programm.
2. Lies Zahl A ein.
3. Lies Zahl B ein.
4. Berechne A + B.
5. Gib das Ergebnis aus.
6. Beende das Programm.

Bewertung: drei sinnvolle Kritikpunkte je 1 P; verbesserter eindeutiger Ablauf bis 3 P.

Aufgabe 2 - 6 Punkte

1. Wiederholung – gleicher Schritt wird mehrfach ausgeführt. – 2 P
2. Entscheidung – Bedingung bestimmt, ob gesprungen wird. – 2 P
3. Sequenz – Eingabe und Ausgabe folgen nacheinander. – 2 P

Aufgabe 3 - 6 Punkte

1. Antwort auf die Frage 3 + 4. – 1 P
2. Antwort = 7. – 1 P
3. „richtig“. – 1 P
4. 1. – 1 P
5. Damit jeder Programmlauf mit einem definierten Punktestand beginnt und kein alter Wert weiterverwendet wird. – 2 P

Aufgabe 4 - 4 Punkte

Beispiel:

```
wiederhole 4 mal  
  gehe n Schritte  
  drehe dich um 90 Grad
```

Algorithmus: 3 P; sinnvoller Vorteil der Schleife (kürzer, übersichtlicher, leichter änderbar): 1 P.

Aufgabe 5 - 4 Punkte

Je Fehlerursache + passender Test bis 2 P. Beispiele:

- Klick-Ereignis fehlt/falsche Figur → Figur anklicken und prüfen, ob Skript ausgelöst wird.
- Variable wird nicht verändert oder um 0 verändert → Block und Wert prüfen.
- Variable wird direkt nach dem Klick wieder auf 0 gesetzt → Reihenfolge beobachten.

Aufgabe 6 - 4 Punkte

Beispiel:

1. Zahl abfragen.
2. Antwort einlesen.
3. Prüfen $\text{Antwort} > 20$.
4. Wenn wahr: „größer als 20“ ausgeben.
5. Sonst: „nicht größer als 20“ ausgeben.

Bewertung nach fachlicher Vollständigkeit und eindeutiger Reihenfolge.